Monte Carlo Foundations: Estimation, Efficiency, and Random Sampling

Roberto Cepeda Rocandio

May 2026; revised October 2026

Abstract

This document opens the notes on Monte Carlo methods. It implements no algorithm: its purpose is to lay the conceptual groundwork on which every Monte Carlo method developed in the companion notes rests. Two themes are developed. First, we formulate derivatives pricing as a numerical integration problem and analyse the standard Monte Carlo estimator: strong consistency, asymptotic normality, confidence intervals, the bias–variance decomposition, and the notion of *computational efficiency* that motivates the entire programme of variance reduction. Second, we examine how independent samples are obtained from prescribed distributions: the desirable properties of pseudorandom number generators, the inverse-transform method as a universal device, and the two standard routes to the normal distribution—inversion and Box–Muller—with attention to the consequences of each choice for the low-discrepancy methods of quasi-Monte Carlo.

Contents

1 Introduction **2**

2 Monte Carlo as Numerical Integration **2**
2.1 The pricing problem as an expectation 2
2.2 The standard Monte Carlo estimator 3
2.3 Convergence rate and the central limit theorem 3
2.4 Confidence intervals from the sample variance 3
2.5 Bias and the mean square error 4
2.6 Computational efficiency of an estimator 4

3 Generating Random Samples **5**
3.1 Pseudorandom number generators 5
3.2 The inverse-transform method 6
3.3 Sampling from the standard normal 7
3.3.1 Inversion: rational approximations of Φ^{-1} 7
3.3.2 Box–Muller and Marsaglia polar 7
3.3.3 Trade-offs and the quasi-Monte Carlo consideration 8
3.4 Reproducibility: seeds and streams 8

4 Roadmap for the Monte Carlo notes **9**

5 Bibliographic notes **10**

1 Introduction

The pricing of a contingent claim under a chosen risk-neutral measure is the computation of an expectation:

$$C \;=\; \mathbb{E}^{\mathbb{Q}}\big[\, e^{-rT}\, \Phi(S_{[0,T]}) \,\big]\,, \tag{1}$$

where Φ is the discounted payoff functional and $S_{[0,T]}$ denotes the relevant trajectory of the underlying state. The notes on Black–Scholes pricing treat the case in which (1) admits a closed form, namely the European call under geometric Brownian motion, where the expectation reduces to a one-dimensional Gaussian integral evaluated analytically through the Black–Scholes formula. We obtain greeks by differentiating that formula, and we recover the implied volatility by inverting it numerically.

That regime is the exception, not the rule. The functional Φ may depend on the entire path (Asian, lookback, barrier options); the underlying may be high-dimensional (basket options, multi-currency products); the dynamics may have no tractable transition density (local volatility, stochastic volatility, jump-diffusion). In all these cases the expectation in (1) must be approximated. Of the available numerical strategies, Monte Carlo simulation is the most widely applicable: it requires only the ability to sample paths of the underlying and evaluate the payoff on those paths.

This series of notes develops Monte Carlo methods systematically, from the elementary estimator for European options to least-squares Monte Carlo for American options. Before any of these algorithms can be implemented or analysed, two prerequisites must be in place. We must understand what a Monte Carlo estimator *is*—what it converges to, how fast, with what uncertainty, and at what cost—and we must understand how the random samples that drive it are produced. This document addresses both. Section 2 treats the estimator. Section 3 treats the samples. Section 4 sketches how the material assembled here threads through the remaining notes on Monte Carlo methods.

We presuppose familiarity with the basic apparatus of probability theory—in particular the law of large numbers, the central limit theorem, and Slutsky's lemma—and with the risk-neutral pricing framework as developed, for instance, in Etheridge [4]. The principal reference for the material in this document is Glasserman [5, Chapters 1 and 2].

2 Monte Carlo as Numerical Integration

2.1 The pricing problem as an expectation

We work on a filtered probability space $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}_{t\in[0,T]}, \mathbb{Q})$ supporting whatever Brownian motions and Poisson random measures the model requires. Under the risk-neutral measure $\mathbb{Q}$, the time-zero price of a European-style contingent claim with $\mathcal{F}_T$-measurable discounted payoff $Y \overset{\text{def}}{=} e^{-rT}\Phi$ is the expectation

$$\alpha \;\overset{\text{def}}{=}\; \mathbb{E}^{\mathbb{Q}}[\, Y \,]. \tag{2}$$

We will treat α throughout as a deterministic real number—the quantity to be estimated—and reserve hats for its estimators. The Black–Scholes framework gives us α in closed form for the European call under Black–Scholes; we recall that $C = S_0\, N(d_1) - Ke^{-rT} N(d_2)$ arises precisely as the evaluation of (2) when S_T is log-normal under $\mathbb{Q}$.

The Monte Carlo approach to (2) is structurally trivial: if we can produce a sequence $Y_1, Y_2, \ldots$ of independent copies of Y, we estimate α by their average. The substance of the method lies entirely in two questions: how good is this estimator, and how do we produce the samples?

2.2 The standard Monte Carlo estimator

Let $Y_1, Y_2, \ldots, Y_n$ be independent and identically distributed copies of Y, with finite second moment. The *Monte Carlo estimator* of α based on n samples is

$$\hat{\alpha}_n \stackrel{\text{def}}{=} \frac{1}{n}\sum_{i=1}^{n} Y_i. \tag{3}$$

Two properties are immediate. First, $\hat{\alpha}_n$ is unbiased: $\mathbb{E}[\hat{\alpha}_n] = \alpha$. Second, by the strong law of large numbers, $\hat{\alpha}_n \to \alpha$ almost surely as $n \to \infty$. The estimator is therefore strongly consistent. Neither property tells us how large n must be to attain a given precision; for that we need the variance of the estimator and its asymptotic distribution.

Writing $\sigma^2 \stackrel{\text{def}}{=} \mathrm{Var}(Y) < \infty$, we have, by independence,

$$\mathrm{Var}(\hat{\alpha}_n) \;=\; \frac{\sigma^2}{n}, \tag{4}$$

so the standard deviation of the estimator decays like $n^{-1/2}$.

2.3 Convergence rate and the central limit theorem

The central limit theorem upgrades (4) to a distributional statement:

$$\sqrt{n}\,(\hat{\alpha}_n - \alpha) \;\xrightarrow{d}\; N(0, \sigma^2) \qquad \text{as } n \to \infty. \tag{5}$$

This is the analytical backbone of Monte Carlo. Three observations are in order.

Order of convergence. The error $|\hat{\alpha}_n - \alpha|$ is, in a probabilistic sense, of order $n^{-1/2}$. To halve the error one must *quadruple* the sample size. This rate is slow when compared with classical quadrature in low dimensions, but it has a feature that classical quadrature lacks: it does not deteriorate with dimension.

Dimension-independence. If $Y = f(U)$ for some $U \in [0,1]^d$ and a sufficiently nice f, the rate in (5) is $n^{-1/2}$ regardless of d. The constant σ does typically depend on d, often unfavourably, but the *order* does not. By contrast, a tensor-product trapezoidal rule on $[0,1]^d$ achieves error $O(n^{-2/d})$ for smooth integrands, which becomes useless even in modest dimensions. This is the structural reason Monte Carlo is the default tool for path-dependent and multi-asset pricing problems: an Asian option monitored at 252 dates is a 252-dimensional integral, and any product rule is hopeless.

Failure modes. The CLT requires $\mathrm{Var}(Y) < \infty$. There are realistic settings where this fails or where it holds but $\mathrm{Var}(Y)$ is so large that the CLT regime is reached only at sample sizes exceeding any plausible computational budget. Heavy-tailed payoffs (e.g. deep out-of-the-money digital options under fat-tailed dynamics) fall into the second category and motivate the importance sampling techniques of variance reduction.

2.4 Confidence intervals from the sample variance

In practice σ is not known a priori. The natural plug-in is the sample standard deviation:

$$S_n^2 \stackrel{\text{def}}{=} \frac{1}{n-1}\sum_{i=1}^{n}(Y_i - \hat{\alpha}_n)^2, \qquad S_n \stackrel{\text{def}}{=} \sqrt{S_n^2}. \tag{6}$$

Since $S_n^2 \to \sigma^2$ almost surely, an application of Slutsky's lemma to (5) yields

$$\frac{\sqrt{n}\,(\hat{\alpha}_n - \alpha)}{S_n} \;\xrightarrow{d}\; N(0,1). \tag{7}$$

For a confidence level $1-\beta$, denoting $z_{\beta/2}$ the upper $\beta/2$-quantile of the standard normal, the asymptotic $(1-\beta)$-confidence interval for α is

$$\hat{\alpha}_n \;\pm\; z_{\beta/2}\,\frac{S_n}{\sqrt{n}}. \tag{8}$$

The half-width

$$h_n \;\stackrel{\text{def}}{=}\; z_{\beta/2}\,\frac{S_n}{\sqrt{n}} \tag{9}$$

is the standard reportable summary of estimator uncertainty in Monte Carlo. It is what every Monte Carlo implementation in these notes computes and returns alongside the point estimate, with one exception: the deterministic quasi-Monte Carlo pricer `mc_european_call_euler_qmc` returns the estimate alone, since a single unrandomised point set provides no sample from which to estimate a half-width.

Remark 2.1. The use of $z_{\beta/2}$ rather than the corresponding t-quantile is purely asymptotic. For n in the thousands, which is the regime of interest throughout these notes, the difference is negligible.

2.5 Bias and the mean square error

The estimator (3) is unbiased by construction, but this exactness will be lost as soon as we replace the exact dynamics of the model with a discretisation, or as soon as we estimate a function of an expectation rather than an expectation itself. Both situations arise in these notes: the former in SDE discretisation (Euler–Maruyama and Milstein schemes), the latter in Longstaff–Schwartz (the nested-expectation structure of American option pricing).

For a generic estimator $\hat{\alpha}$ of a target α, the mean square error decomposes as

$$\mathrm{MSE}(\hat{\alpha}) \;=\; \mathbb{E}[(\hat{\alpha}-\alpha)^2] \;=\; \big(\mathbb{E}[\hat{\alpha}]-\alpha\big)^2 + \mathrm{Var}(\hat{\alpha}) \;=\; \mathrm{Bias}^2(\hat{\alpha}) + \mathrm{Var}(\hat{\alpha}). \tag{10}$$

When discretisation enters, $\hat{\alpha}$ depends on a step size Δt, and a tradeoff appears: refining Δt reduces bias but, for a fixed computational budget, also reduces the number of replications and hence increases variance. The MSE-optimal choice of Δt balances the two contributions and is derived explicitly in the note on the Euler–Maruyama pricer.

2.6 Computational efficiency of an estimator

The remaining piece of conceptual scaffolding is the notion of *efficiency*, which underpins the entire programme of variance reduction.

Suppose two unbiased estimators $\hat{\alpha}$ and $\hat{\alpha}'$ of the same quantity α are available, with per-replication variances σ^2 and σ'^2 and per-replication computational costs τ and τ' respectively. Naively, one prefers the estimator with smaller variance. This is incorrect: what matters is the total computational work required to reach a prescribed precision.

Proposition 2.2 (Efficiency comparison)**.** *Let $\hat{\alpha}$ and $\hat{\alpha}'$ be unbiased estimators of α with per-replication variances σ^2, σ'^2 and costs τ, τ'. Fix a target half-width ε for the asymptotic confidence interval (8) at confidence level $1-\beta$. Then $\hat{\alpha}$ requires less total computational work than $\hat{\alpha}'$ to attain the prescribed half-width if and only if*

$$\sigma^2\tau \;<\; \sigma'^2\,\tau'. \tag{11}$$

Proof. For an estimator with per-replication variance σ^2, attaining half-width ε requires, asymptotically, $n_\varepsilon = (z_{\beta/2}\,\sigma/\varepsilon)^2$ replications by (8). The corresponding total cost is $n_\varepsilon \cdot \tau = (z_{\beta/2}/\varepsilon)^2\sigma^2\tau$. The factor $(z_{\beta/2}/\varepsilon)^2$ does not depend on the estimator, so the comparison reduces to that of the products $\sigma^2\tau$ and $\sigma'^2\,\tau'$. □

Definition 2.3. The *computational efficiency* of an unbiased estimator with variance σ^2 and per-replication cost τ is $(\sigma^2\tau)^{-1}$.

Remark 2.4 (Why this matters). Variance reduction techniques almost always increase the per-replication cost: an antithetic estimator generates one extra path per pair, a control variate evaluates an additional payoff, importance sampling involves an extra likelihood ratio, and so on. The relevant question is not whether the variance went down, but whether the product $\sigma^2\tau$ went down. The variance-reduction notes compare estimators at equal payoff count, so their reported factors measure the variance alone; the cost factor τ is not measured, and a reported variance reduction is a gain in efficiency only if it exceeds the ratio of per-replication costs.

Remark 2.5 (Biased estimators). Proposition 2.2 assumes both estimators are unbiased. For estimators carrying bias b controlled by some parameter (typically a discretisation step), the analogous comparison balances MSE against work, and the MSE-optimal allocation between bias and variance involves the asymptotic rates of b and σ as functions of the parameter. A representative analysis of this kind is carried out in the note on the Euler–Maruyama pricer.

3 Generating Random Samples

The estimator (3) requires independent copies of Y. The standard pipeline for producing them is two-stage. First, a *pseudorandom number generator* produces a deterministic sequence $u_1, u_2, \cdots \in [0, 1)$ designed to be statistically indistinguishable, for the purposes of the simulation, from i.i.d. samples of the uniform distribution on $[0, 1]$. Second, a *sampling transformation* maps these uniforms into samples from the distributions actually required by the model: standard normals for Brownian increments, exponentials for Poisson interarrivals, and so on. Both stages admit several constructions, and the choices interact in ways that matter beyond the present note.

3.1 Pseudorandom number generators

A pseudorandom number generator (PRNG) is a deterministic algorithm that produces, from an initial *seed*, a long sequence of values in $[0, 1)$ which passes a battery of statistical tests for uniformity and independence. The output is reproducible: the same seed yields the same sequence. This determinism is a feature, not a defect: it is what makes debugging, regression testing, and exact reproduction of numerical results possible.

The minimum properties demanded of a PRNG suitable for serious simulation are conventionally:

1. **Long period.** The eventual cycle length should exceed by several orders of magnitude the largest sample size that will be consumed.

2. **High-dimensional equidistribution.** For applications that form d-tuples of consecutive outputs and treat them as points in $[0, 1)^d$ (which is exactly what we shall do every time we sample a multivariate normal), the empirical distribution of these d-tuples should be uniform in $[0, 1)^d$ for d as large as is relevant.

3. **Speed and small state.** Generation must be fast, and the internal state must be small enough to fit comfortably in cache.

4. **Statistical quality.** The output must pass standard empirical test suites (TestU01's BigCrush is the contemporary benchmark).

What not to use. The classical *linear congruential generator* (LCG) defined by $X_{n+1} = (aX_n + c) \bmod m$, normalised to $u_n = X_n/m$, is instructive as a cautionary example. Marsaglia showed in 1968 that all d-tuples of consecutive outputs of an LCG lie on at most $(d!\,m)^{1/d}$ parallel hyperplanes in $[0,1)^d$, an arrangement easily detected by any test sensitive to multidimensional structure. LCGs with poorly chosen parameters were standard in the 1970s and 1980s and contaminated a great deal of simulation work; some remain bundled with libraries to this day. They have no place in financial Monte Carlo.

Acceptable defaults. Two families of generators are in widespread use and meet the criteria above:

- The *Mersenne Twister* (MT19937) of Matsumoto and Nishimura [7] has period $2^{19937} - 1$ and is 623-distributed to 32-bit accuracy. It is the default in NumPy's legacy `RandomState`, and the C++ standard library provides it as `std::mt19937` and `std::mt19937_64` (the library's `std::default_random_engine` is implementation-defined, so the project names `std::mt19937_64` explicitly).
- The *permuted congruential generator* family (PCG) of O'Neill [9] couples an LCG with a non-linear output permutation, achieving statistical quality comparable to MT19937 with a much smaller state and faster generation. NumPy's modern `Generator` interface uses PCG64 by default.

For this project we adopt PCG64 in Python (the NumPy default) and `std::mt19937_64` in C++. The choice is not critical at our sample sizes; either generator is comfortably above the quality threshold required.

True randomness. Hardware-based true random number generators exist and are appropriate for cryptography, but they are unsuitable for simulation precisely because their output cannot be reproduced. A simulation result that depends on a true random sequence is not falsifiable.

3.2 The inverse-transform method

Given uniforms, we now turn to producing samples from arbitrary prescribed distributions. The most general and conceptually cleanest method is inversion.

Theorem 3.1 (Inverse-transform sampling)**.** *Let $F : \mathbb{R} \to [0,1]$ be a cumulative distribution function, and define its generalised inverse*

$$F^{-1}(u) \stackrel{\text{def}}{=} \inf\{x \in \mathbb{R} : F(x) \geq u\}, \qquad u \in (0,1). \tag{12}$$

If $U \sim U(0,1)$, then $X \stackrel{\text{def}}{=} F^{-1}(U)$ has distribution function F.

Proof. By construction, $F^{-1}(u) \leq x$ if and only if $u \leq F(x)$. Hence $\mathbb{P}(X \leq x) = \mathbb{P}(F^{-1}(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x)$. □

When F is continuous and strictly increasing, F^{-1} is the ordinary inverse function. When F has flats (corresponding to gaps in the support of the distribution) or jumps (corresponding to atoms), the generalised inverse handles both correctly, and the theorem applies without modification.

The practical bottleneck of inversion is the evaluation of F^{-1}. For some distributions it is closed-form (the exponential: $F^{-1}(u) = -\lambda^{-1}\log(1-u)$); for the most important distribution in finance, the standard normal, it is not, and must be approximated. Section 3.3 addresses this.

Why inversion is preferred over alternatives. Two properties recommend inversion as the default method.

First, inversion is *coordinate-preserving*: it consumes one uniform and produces one sample of the target distribution. This means that if $(U_1, \dots, U_d)$ is a sample from a d-dimensional uniform construction, then $(F^{-1}(U_1), \dots, F^{-1}(U_d))$ is the corresponding sample of independent variables with distribution F, and the geometric structure of the d-dimensional construction is preserved. This property is essential for quasi-Monte Carlo, the subject of the note on quasi-Monte Carlo foundations, where the points $(U_1, \dots, U_d)$ are drawn from a low-discrepancy sequence designed to fill $[0,1)^d$ uniformly. Methods that consume two uniforms per sample (such as Box–Muller, see below) mix two coordinates of the construction together and destroy this structure.

Second, inversion is *monotone* in U, which makes it compatible with stratification and with antithetic sampling: the antithetic of $X = F^{-1}(U)$ is $X' = F^{-1}(1-U)$, which is the correct antithetic in the sense of being perfectly negatively correlated with X when F is symmetric.

3.3 Sampling from the standard normal

Brownian increments are by far the most common random ingredient of financial Monte Carlo, so the efficient and accurate generation of standard normal samples deserves dedicated attention.

3.3.1 Inversion: rational approximations of Φ^{-1}

The standard normal inverse Φ^{-1} has no elementary closed form. What is available, and what every numerical library uses internally, is a rational approximation to Φ^{-1} of sufficiently high accuracy that the residual error is irrelevant compared with Monte Carlo sampling error. The classical approximation is Beasley–Springer [3] as extended by Moro [8]; Wichura's algorithm AS 241 [10] is a widely used alternative; Acklam's approximation [1] has relative error below 1.15×10^{-9} throughout the support, and reaches full double precision only after an additional refinement step. SciPy's `scipy.special.ndtri`, which underlies `scipy.stats.norm.ppf`, comes from the Cephes library.

In Python the Monte Carlo implementations call `scipy.stats.norm.ppf`. The C++ code has no Boost dependency: it uses its own implementation of Acklam's approximation (`inverse_normal_cdf` in `gbm.cpp`), without the refinement step, so its quantiles carry a relative error below 1.15×10^{-9}, far below Monte Carlo sampling error. The conceptual point is that inversion-based normal sampling is a single call to a transcendental function: it is a few times slower than the alternative below, and that cost is the price paid for compatibility with quasi-Monte Carlo.

3.3.2 Box–Muller and Marsaglia polar

An elegant alternative converts pairs of uniforms directly into pairs of independent standard normals, without ever computing Φ^{-1}.

Theorem 3.2 (Box–Muller transformation)**.** *Let $U_1, U_2 \sim U(0,1)$ be independent. Define*

$$X_1 = \sqrt{-2\log U_1}\,\cos(2\pi U_2), \tag{13}$$

$$X_2 = \sqrt{-2\log U_1}\,\sin(2\pi U_2). \tag{14}$$

Then X_1 and X_2 are independent standard normal random variables.

Proof. Writing $R = \sqrt{X_1^2 + X_2^2}$ and $\Theta = \arctan(X_2/X_1)$, the joint density of (R, Θ) that would arise from $(X_1, X_2) \sim N(0, I_2)$ in polar coordinates is

$$f_{R,\Theta}(r,\theta) \;=\; \frac{1}{2\pi}\, r\, e^{-r^2/2}, \qquad r \geq 0,\ \theta \in [0, 2\pi),$$

which factors as the product of an exponential density on $r^2/2$ and a uniform density on $[0, 2\pi)$. Equivalently, with $V = R^2/2 \sim \text{Exp}(1)$ and $\Theta \sim U[0, 2\pi)$ independent, $(X_1, X_2) = (\sqrt{2V}\cos\Theta, \sqrt{2V}\sin\Theta)$ is standard bivariate normal. Sampling $V = -\log U_1 \sim \text{Exp}(1)$ by inversion of the exponential and $\Theta = 2\pi U_2$ by linear scaling of the uniform recovers (13). □

The Marsaglia polar method is a refinement that replaces the trigonometric calls with rejection sampling on the unit disc, and is marginally faster on architectures with slow sin and cos. The distinction is irrelevant on modern hardware.

3.3.3 Trade-offs and the quasi-Monte Carlo consideration

Box–Muller and inversion produce statistically equivalent samples for the purposes of classical Monte Carlo. They differ in three respects:

1. **Speed.** Box–Muller (or Marsaglia polar) is moderately faster than inversion, since rational approximation of Φ^{-1} is costlier than $\sqrt{}, \log, \cos, \sin$.

2. **Coordinate use.** Box–Muller consumes two uniforms to produce two normals. Inversion consumes one uniform to produce one normal.

3. **Compatibility with low-discrepancy sequences.** A quasi-MC construction populates $[0, 1)^d$ with deterministic points designed to have low discrepancy in each coordinate. If we transform these via inversion coordinatewise, the low-discrepancy property survives. If we transform them via Box–Muller, we mix coordinates 1 and 2 nonlinearly through the trigonometric pair, and the equidistribution property is destroyed in those coordinates. Box–Muller is therefore incompatible with quasi-Monte Carlo.

For this reason the C++ code samples every normal by inversion, and none of the code uses Box–Muller. In Python, inversion is used by the exact GBM sampler and the code built on it (the Greeks, the antithetic and control-variate pricers, the Heston simulators) and by the quasi-Monte Carlo pricers; the Euler and Milstein path samplers (`rng.normal` in `quantlib.gbm`), the Asian pricers and the Longstaff–Schwartz pricer (`rng.standard_normal`) use NumPy's ziggurat sampler instead. Quasi-Monte Carlo exploits inversion directly.

3.4 Reproducibility: seeds and streams

Every Monte Carlo result reported in this project must be reproducible from a recorded seed. This is non-negotiable for two reasons. First, debugging a wrong result is impossible if the wrong result cannot be reproduced; an off-by-one error in an estimator has to be examined on a fixed sequence of paths. Second, regression testing requires that results not change unexpectedly between commits, which is impossible without seed control.

In Python, this means consistently constructing generators via `rng = numpy.random.default_rng(seed)` with an explicit integer `seed`, and passing `rng` as a parameter to every function that consumes randomness. The use of the deprecated module-level functions `numpy.random.normal` et al., which implicitly draw from a global mutable state, is forbidden in this codebase: it makes thread-safety unattainable and makes the order in which functions are called silently affect their outputs.

In C++, the analogous discipline is to declare an `std::mt19937_64 rng(seed)` as a local variable in the entry point of a simulation and pass `rng` by reference to every function that consumes randomness. Each function draws uniforms through a locally constructed `std::uniform_real_distribution` object and maps them to normals by inversion; the library's `std::normal_distribution`, which may cache a value between calls, is not used. The engine itself is owned by exactly one stack frame and threaded through.

When variance reduction enters, certain techniques (in particular common random numbers, used pervasively in greek estimation) require that two related estimators consume the *same* sequence of uniforms. The seed-and-pass-by-reference discipline established now makes this trivial; an alternative discipline based on global state makes it impossible.

4 Roadmap for the Monte Carlo notes

The material assembled in this document threads through the remaining notes on Monte Carlo methods as follows.

Vanilla Monte Carlo and SDE discretisation. Section 2 provides directly the framework for the European call MC pricer: an estimator of the form (3) applied to discounted payoffs, with the half-width (9) as the precision report. The discretisation schemes introduce bias for the first time, and the bias–variance decomposition (10) of Section 2.5 becomes the analytical tool for choosing time steps.

Variance reduction. Every variance-reduction method—antithetic variates, control variates, stratified sampling, importance sampling—is justified through the efficiency criterion of Proposition 2.2: the variance reduction must dominate the additional per-replication cost. The monotonicity property of inversion (Section 3.2) underlies the antithetic construction; the seed-control discipline of Section 3.4 is required for control variates and for the common-random-numbers technique used in conjunction with greeks.

Quasi-Monte Carlo. The use of inversion as the sampling transformation (Section 3.3) is motivated by quasi-Monte Carlo: low-discrepancy sequences cannot be processed through Box–Muller without losing their structure. The asymptotic regime of (5) is replaced by a deterministic Koksma–Hlawka bound, which improves the order of convergence from $n^{-1/2}$ to nearly n^{-1} at the cost of dimension dependence in the constant.

Greeks. Pathwise and likelihood-ratio estimators of price sensitivities are themselves Monte Carlo estimators, to which the apparatus of Section 2 applies directly. Bumped finite-difference estimators introduce both bias (from the finite-difference truncation) and variance, and the optimal bump size balances them according to the same logic as the discretisation step of the SDE schemes. Common random numbers across the bumped and unbumped simulations are essential for variance control.

Path-dependent options. Asian, barrier, and lookback payoffs are functionals of the entire simulated path. The machinery of estimator, variance, and confidence interval is unchanged; the per-replication cost increases, and the variance can be very large (barrier options especially), making variance reduction techniques not optional but essential.

American options. Least-squares Monte Carlo introduces a nested estimation structure—continuation values are themselves estimated within a Monte Carlo loop—which produces a bias that does not vanish as the outer sample size grows. The bias–variance framework of Section 2.5 is again the relevant analytical tool, now with a non-trivial bias contribution that must be controlled by basis selection rather than by sample size alone.

5 Bibliographic notes

The principal reference throughout the Monte Carlo notes is Glasserman [5]; the material of this document corresponds to Sections 1.1 and Chapter 2 of that text, with a slight reordering to bring the efficiency criterion into the same section as the basic estimator analysis. Asmussen and Glynn [2] provide a more probabilistic and less finance-oriented treatment of the same material, with a substantially more detailed discussion of estimator asymptotics and of random number generation; their Chapter 3 is the recommended supplement for the contents of Section 2, and their Chapter 2 covers the RNG material of Section 3.1 in greater depth. For PRNGs specifically, L'Ecuyer [6] is a thorough survey from one of the leading practitioners in the field; the original papers on the Mersenne Twister [7] and the PCG family [9] are short and readable.

References

[1] P. J. Acklam, *An algorithm for computing the inverse normal cumulative distribution function*, Available online (technical note), 2003.

[2] S. Asmussen and P. W. Glynn, *Stochastic Simulation: Algorithms and Analysis*, Springer, 2007.

[3] J. D. Beasley and S. G. Springer, *The percentage points of the normal distribution*, Applied Statistics, 26(1):118–121, 1977.

[4] A. Etheridge, *A Course in Financial Calculus*, Cambridge University Press, 2002.

[5] P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer, 2004.

[6] P. L'Ecuyer, *Random Number Generation*, in: J. E. Gentle, W. K. Härdle, Y. Mori (eds.), Handbook of Computational Statistics, Springer, 2012, pp. 35–71.

[7] M. Matsumoto and T. Nishimura, *Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator*, ACM Transactions on Modeling and Computer Simulation, 8(1):3–30, 1998.

[8] B. Moro, *The full Monte*, Risk, 8(2):57–58, 1995.

[9] M. E. O'Neill, *PCG: A family of simple fast space-efficient statistically good algorithms for random number generation*, Technical Report HMC-CS-2014-0905, Harvey Mudd College, 2014.

[10] M. J. Wichura, *Algorithm AS 241: The percentage points of the normal distribution*, Applied Statistics, 37(3):477–484, 1988.